

TalentMesh

Software Engineering Project Report

The Documentation of the TalentMesh Platform

Prepared by

Taha Karagöz

Zeynep Nur Poyraz

Arda Doğan

Kerem Sönmez

Sena Çelik

Yusuf Berk Aksoy

June 2026

Table of Contents

I. Project Description

1. Project Overview
2. The Purpose of the Project
3. The Scope of the Work
4. Product Scenarios
5. Stakeholders
6. Mandated Constraints
7. Naming Conventions and Definitions
8. Relevant Facts and Assumptions

II. Requirements

9. Product Use Cases
 10. Functional Requirements
 11. Data Requirements
 12. Dependability Requirements
- ## III. Non-Functional Requirements
13. Maintainability and Supportability
 14. Security Requirements
 15. Usability and Humanity Requirements
 16. Look and Feel Requirements
 17. Operational and Environmental Requirements
 18. Cultural, Political, and Legal Requirements

IV. Design & Test Plans

19. System Design Goals
20. Software Architecture
21. Test Plans

V. Project Issues

22. Open Issues and Risks

VI. Glossary

VII. References

I. Project Description

1. Project Overview

TalentMesh is a cross-platform mobile and web application built with Flutter and Supabase. It serves as a professional networking and team-building platform aimed at university students and early-career software professionals. Users can create detailed profiles showcasing their skills and education, discover and connect with peers, form project teams, manage collaborative tasks and meetings, and endorse one another's skills. The platform bridges the gap between academic life and professional collaboration by providing a structured environment for project-based networking.

2. The Purpose of the Project

2a. User Business and Background

University students and junior developers frequently struggle to find suitable project partners with complementary skill sets. Existing platforms such as LinkedIn are too formal and not optimized for short-term academic or side projects. WhatsApp groups and Discord servers lack structured discovery tools, and Instagram provides no professional context.

As a result, talented individuals remain isolated within their departments, unable to form interdisciplinary teams. TalentMesh addresses this fragmentation by providing a dedicated, lightweight social platform where users can present their competencies, express the roles they seek, and discover peers who match their needs.

2b. Goals of the Project

The project is organized around the following functional modules:

- **Authentication Module:** Secure sign-up, email verification, login, and password reset flows.
- **Profile Management Module:** Rich user profiles with full name, username, avatar, school, department, degree, year, skills, looking-for preferences, open-to-work flag, and bio.
- **Social / Connection Module:** Friend request system with accept/reject/remove mechanics, a friends list, and a teammate tracking screen.
- **Feed Module:** A social feed where users can post updates visible to their network.
- **Search & Discovery Module:** Full-text and skill-based search across profiles, with endorsement-weighted ranking.
- **Team Management Module:** Create teams with defined roles, required skills, and capacity limits; manage join requests; view team details.
- **Task Management Module:** Kanban-style task board (Todo / In Progress / Done) with subtasks, assignees, and due dates.
- **Meeting Module:** Schedule team meetings with date/time, description, and an optional link; automated in-app notifications.
- **Notification Module:** In-app notification center covering friend requests, team invitations, task assignments, and meeting alerts.

- **Settings Module:** Profile visibility, notification preferences, language switching (TR/EN), and security settings.

2c. Measurement

Project success will be evaluated through the following metrics:

- User Retention: percentage of registered users who log in at least once per week after the first month.
- Team Formation Rate: number of teams created and filled to at least 50% capacity per month.
- Connection Growth: average number of accepted friend connections per active user.
- Task Completion Rate: percentage of tasks marked Done within their due date.
- User Satisfaction Score: in-app rating prompt (1–5 stars) after first 7 days of use; target average ≥ 4.0 .
- Notification Engagement Rate: percentage of in-app notifications that result in a follow-up action within 24 hours.

3. The Scope of the Work

3a. The Current Situation

University students currently rely on informal channels to find project collaborators. None of the available tools provide a unified profile, discovery, and team management workflow within a single application:

Platform / Method	Key Weakness
WhatsApp / Telegram groups	No skill filtering, ephemeral history, high noise
LinkedIn	Designed for job hunting, not short-term academic project teaming
Discord servers	No structured profiles, no discovery or matching engine
GitHub	Code-only; no social or real-time collaboration layer
Departmental bulletin boards	Physical or digital boards with no matching algorithm

3b. The Context of the Work

The system interacts with the following actors and organizational entities:

Actor	Role
Registered User	Creates profile, connects with peers, joins or creates teams
Team Admin	Creates and manages a team, approves join requests, assigns tasks
Team Member	Participates in tasks and meetings within a team
System (Supabase)	Handles authentication, data persistence, file storage, and real-time events
Notification Service	Sends in-app alerts triggered by user actions

3c. Work Partitioning — Event List

#	Event Name	Input	Output	Summary
1	User Registration	Email, password, full name	Verified account; profile row created	New user completes sign-up and email verification; profile initialized in profiles table
2	Send Friend Request	Requester ID, target user ID	Pending entry in friend_requests; notification sent	User discovers another user via search and initiates a connection
3	Create Team	Team name, description, roles/skills, max members	Team row in teams; creator set as admin	User creates a new project team and publishes it for discovery
4	Send Join Request	Team ID, user ID	Pending entry in team_requests; admin notified	A user applies to join a discovered team
5	Accept Join Request	Request ID, team ID, user ID	team_members row created; applicant notified	Team admin approves a membership application after capacity check
6	Create Task	Team ID, title, assignees, due date, subtasks	Row in team_tasks; assignees notified	Admin or member creates a task and assigns it to team members
7	Schedule Meeting	Team ID, title, date, link	Row in team_meetings; all members notified	A team member schedules a meeting; reminder service polls for upcoming events
8	Endorse Skill	Endorser ID, target user ID, skill name	Row inserted/deleted in skill_endorsements	A friend toggles an endorsement on one of the target user's listed skills

3d. Competing Products

Competitor	Category	TalentMesh Differentiator
LinkedIn	Professional networking	Targets students with lightweight profiles, no recruiters, project-first mindset
GitHub	Code collaboration	Adds social discovery, messaging, and support for non-technical roles
Slack / Discord	Team communication	Provides built-in team formation, task boards, and skill matching
Handshake	Campus recruiting	Focuses on peer collaboration rather than employer-to-student pipelines
Meetup	Event-based networking	Persistent and project-centric, not event-driven

TalentMesh's unique value proposition: skill-based smart matchmaking combined with in-app team lifecycle management — covering discovery, joining, task tracking, and meetings — in a single mobile-first Flutter application.

4. Product Scenarios

4a. Product Scenario List

- New user registers, verifies email, and completes their profile with skills and department.
- User searches for peers by skill keyword and sends a friend request.
- User creates a team specifying required roles and skills, then reviews incoming join requests.
- A prospective member discovers the team via Smart Match, views team details, and applies.
- Team admin accepts the request; the new member receives an in-app notification.
- Team admin creates a task and assigns it to two members.
- Members update task status on the Kanban board as work progresses.
- Team admin schedules a weekly meeting; all members receive an in-app notification.
- Meeting reminder fires 30 minutes before the scheduled time.
- Users endorse each other's skills, increasing their visibility in search results.

4b. Individual Product Scenarios

Daily Operations:

Each morning, a team admin reviews the Kanban board, reassigns any overdue tasks, and checks the notification center for new join requests. During the day, team members move tasks from In Progress to Done and log brief status updates. Notifications are delivered in real-time via Supabase Realtime subscriptions.

Weekly Operations:

At the start of each week, the admin schedules the upcoming team meeting through the meeting scheduler. Members receive an in-app notification with the date, time, and link. A reminder notification fires 30 minutes before the meeting. After the meeting, the admin creates new tasks based on decisions made and assigns them to members.

5. Stakeholders

5a. The Client

The client for TalentMesh is the project development team itself, acting as both developers and primary users during the academic pilot phase. The platform is designed for student clubs and university communities that require a private, structured collaboration environment.

5b. The Customer

The primary customer segment is university students seeking project partners and early-career software professionals looking to build a portfolio through collaborative side projects. Student organizations at other universities are also prospective customers.

5c. Hands-On Users of the Product

Characteristic	Description
User Category	University students, junior developers, recent graduates

Age Group	18–26
Technical Experience	Moderate to high; comfortable with mobile applications
Education Level	Undergraduate or higher
Languages	Turkish and English (app supports both)
Device Preference	Primarily Android smartphones; secondary iOS and web browser
Motivation	Finding project partners, building a portfolio, gaining team experience

5d. Priorities Assigned to Users

Priority Level	User Type	Access Scope
Key User	Registered active user with a complete profile	Full feature access across all modules
Secondary User	Registered user with an incomplete profile	Limited discovery; cannot create teams
Unimportant User	Unauthenticated visitor	No access; redirected to login / sign-up screen

Profile completeness gates certain features: a user without listed skills will not appear in Smart Match recommendations.

5e. User Participation

The following engagement mechanisms are implemented to sustain user activity:

- **Skill Endorsement System:** Friends can endorse each other's skills; endorsed skills gain a visual indicator and a ranking boost in search results.
- **Open to Work Badge:** Displayed prominently on profile cards to signal availability to potential teammates.
- **Meeting Reminders:** Automated in-app alerts keep users engaged with their team schedule.
- **Notification Center:** Aggregated activity feed ensures users return to act on pending requests and assignments.
- **Smart Match Feed:** Regularly refreshed team suggestions based on skill overlap encourage exploration.

5f. Maintenance Users and Service Technicians

Role	Responsibility
Developer / Admin	Supabase dashboard access for schema migrations, RLS policy updates, storage bucket management
Flutter Developer	App updates pushed via app store or internal distribution
Database Admin	Index optimization, backup schedule management, slow query resolution

6. Mandated Constraints

6a. Solution Constraints

Constraint 1 — Authentication

Description: All users must authenticate via Supabase Auth (email/password) before accessing any protected screen.

Rationale: Prevents unauthorized access to user data and ensures Row-Level Security policies can enforce per-user data isolation.

Fit Criterion: Any API call to profiles, teams, or messaging tables without a valid JWT returns HTTP 401. The AuthGate widget redirects unauthenticated users to the login screen.

Constraint 2 — Cross-Platform Flutter Build

Description: The application must compile and run on Android (primary), iOS, and Web without platform-specific feature degradation.

Rationale: The student user base uses diverse devices; a single codebase reduces maintenance overhead.

Fit Criterion: flutter build apk, flutter build ios, and flutter build web all complete without errors on the reference CI machine.

Constraint 3 — Supabase as Sole Backend

Description: All data persistence, authentication, file storage, and real-time subscriptions must use Supabase services exclusively.

Rationale: Eliminates the need for a separate server, reduces operational complexity, and leverages Supabase's built-in RLS for security.

Fit Criterion: No third-party server-side code is deployed; all database operations use supabase_flutter ^2.9.0.

Constraint 4 — Internationalization

Description: All UI strings must be externalized via easy_localization JSON files for Turkish and English.

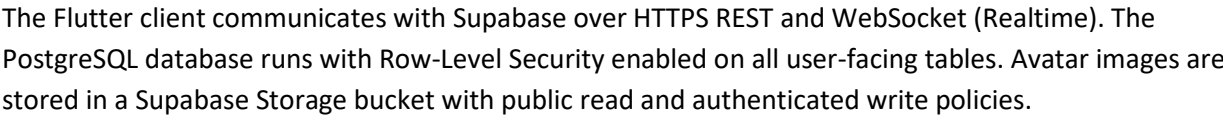
Rationale: The target user base spans both Turkish-speaking and English-speaking environments.

Fit Criterion: Switching locale in Settings instantly updates all displayed strings without an app restart; no hardcoded strings exist in widget build methods.

6b. Implementation Environment

The system infrastructure is structured as follows:

```
User Device (Android / iOS / Web Browser)
      |
      ▼
```

- Users access the app primarily on personal Android smartphones running Android 10 or later.
- A stable internet connection (4G or Wi-Fi) is required for all data operations; offline mode is not supported in v1.
- The app is expected to be used in university campuses, dormitories, and home environments.
- Minimum device requirements: 2 GB RAM, 100 MB free storage.

Term	Definition
Profile	A user's public-facing data record in the profiles table, including full name, username, avatar URL, skills array, department, school, degree, education year, bio, and visibility settings.
Skill Endorsement	A directed affirmation from one user (endorser) to another (endorsee) confirming a specific skill, stored in skill_endorsements. Only friends can endorse each other.
Smart Match	An algorithmic recommendation that surfaces teams whose required_skills intersect with the current user's skills array, excluding teams the user administrates.
Team Admin	The user who created a team (admin_id in teams table). Approves join requests, manages members, creates tasks and meetings, and can delete the team.
Open to Work	A boolean flag (open_to_work) on the user's profile signaling availability for new projects, displayed as a badge on profile cards in search results.
Kanban Board	The task management view within a team detail screen, organizing TeamTask records into three swim lanes: To-Do, In Progress, and Done.

- Flutter SDK version ^3.10.1 and Dart SDK are required on developer machines; older versions are not supported.

- Supabase free tier limits concurrent connections to 60 and database size to 500 MB; exceeding these requires a paid plan.
- easy_localization ^3.0.8 persists the selected locale in SharedPreferences; no additional persistence code is required.
- image_picker ^1.2.1 requires CAMERA and READ_EXTERNAL_STORAGE permissions declared in AndroidManifest.xml.
- Row-Level Security policies must be explicitly enabled per table; a missing policy defaults to no access.

Assumptions:

- The user base will not exceed 1,000 concurrent active users during the academic pilot phase; Supabase free tier is sufficient.
- All users possess a valid university or personal email address for account registration.
- Future releases may introduce push notifications (FCM); the current implementation uses in-app notifications only.
- KVKK compliance will be addressed before any public release by adding a privacy policy consent screen.
- Data volume per user is expected to grow by approximately 20% per semester.

II. Requirements

9. Product Use Cases

Actor: User (Authenticated, Regular)

Use Case ID	Use Case Name	Description
UC-01	Register	User creates a new account with email and password; receives a verification email
UC-02	Login	User authenticates with email and password; redirected to feed on success
UC-03	Reset Password	User requests a password reset email from the forgot-password screen
UC-04	View & Edit Profile	User views own profile; edits bio, skills, education, avatar, and preferences
UC-05	Search Users	User searches for other users by name, skill, or department
UC-06	Send Friend Request	User sends a connection request to another user's profile
UC-07	Accept / Reject Friend Request	User responds to an incoming friend request
UC-08	Remove Friend	User removes an existing friend connection
UC-09	Endorse Skill	User endorses a skill on a friend's profile (friends only)
UC-10	View Feed	User browses the social post feed
UC-11	Create Post	User publishes a text post to the feed
UC-12	Browse Teams	User discovers teams via search or Smart Match recommendations
UC-13	Send Join Request	User applies to join an open team
UC-14	View Team Detail	User views team description, required roles/skills, members, tasks, and meetings
UC-15	View Notifications	User reads and acts on in-app notifications
UC-16	Change Settings	User updates language, visibility, notification preferences, or password
UC-17	View Teammates	User views active and former project teammates
UC-18	Chat	User sends real-time direct messages to a connection

Actor: Team Admin (Authenticated, Elevated Within Team Context)

Use Case ID	Use Case Name	Description
UC-19	Create Team	Admin creates a new team with name, description, required roles/skills, and capacity
UC-20	Review Join Requests	Admin views pending applications to the team
UC-21	Accept Join Request	Admin accepts an applicant (capacity checked)

UC-22	Reject Join Request	Admin declines an applicant
UC-23	Remove Team Member	Admin removes a member from the team
UC-24	Update Team Description	Admin edits the team description
UC-25	Delete Team	Admin permanently deletes the team and all associated records
UC-26	Create Task	Admin creates a task, assigns it to members, sets due date and subtasks
UC-27	Update Task Status	Admin or member moves a task between Kanban columns
UC-28	Delete Task	Admin deletes a task
UC-29	Schedule Meeting	Admin creates a meeting entry with date, link, and description
UC-30	Delete Meeting	Admin removes a scheduled meeting

10. Functional Requirements

FR-01 — User Authentication

Description: The system shall authenticate users using email and password via Supabase Auth. Users who have not verified their email shall be redirected to the verification screen upon login attempt.

Rationale: Authentication is the gateway to all protected data. Email verification ensures valid accounts and reduces spam profiles.

Fit Criterion: A user providing valid credentials with a verified email is granted access to the main feed within 3 seconds. A user with an unverified email is shown the verification screen with a resend option.

FR-02 — Profile Management

Description: The system shall allow users to create and update their profile including full name, username, avatar, bio, department, school, degree, education year, skills array, looking-for preferences, and open-to-work flag.

Rationale: A rich, accurate profile is the foundation of meaningful skill-based matching and discovery.

Fit Criterion: Profile changes are persisted to the profiles table via upsert within 5 seconds. The updated avatar URL is reflected in the UI immediately after upload.

FR-03 — Friend Connection System

Description: The system shall allow users to send, accept, and reject friend requests. Accepted friends appear in each other's friend list. Users can remove a friend, which deletes or marks the friend_requests row.

Rationale: The friend graph gates skill endorsements and determines the social context of the feed.

Fit Criterion: A sent request creates a pending row in friend_requests. Accepting creates an accepted row and triggers a notification. Removing deletes the row, and the user disappears from both parties' friend lists.

FR-04 — Team Creation and Management

Description: The system shall allow authenticated users to create teams specifying a name, description, required roles, required skills, and maximum member count. The creator is automatically assigned as admin_id.

Rationale: Teams are the core collaborative unit of the platform; structured metadata enables smart matching.

Fit Criterion: A created team appears in the My Teams screen within 2 seconds. Deleting a team cascades removal of associated records (tasks, meetings, membership rows) via Supabase cascade deletes.

FR-05 — Smart Match

Description: The system shall present a list of up to 5 teams whose required_skills array intersects with the current user's skills array. Teams administered by the current user are excluded.

Rationale: Reduces discovery friction by surfacing relevant teams automatically.

Fit Criterion: The smart match list is non-empty for any user with at least one skill appearing in any team's required_skills. Results update within 3 seconds of opening the relevant screen.

FR-06 — Task Management (Kanban)

Description: Within a team, the admin or any member shall be able to create tasks with a title, description, due date, subtask list (JSONB), and one or more assignees. Task status transitions across three states: todo, in_progress, and done.

Rationale: Provides lightweight project management within the team context, reducing reliance on external tools.

Fit Criterion: A created task appears in the Todo column immediately. Status updates are persisted to the team_tasks table. Assigned users receive an in-app notification.

FR-07 — Meeting Scheduling

Description: The system shall allow team members to schedule meetings with a title, date/time, optional description, and optional URL. A background reminder service shall poll for meetings starting within 30 minutes and trigger an in-app notification.

Rationale: Keeps team members aligned on synchronous communication points without requiring an external calendar.

Fit Criterion: A created meeting appears in the team's meeting list sorted by date. The MeetingReminderService fires a notification to all team members at most 30 minutes before the meeting.

FR-08 — Skill Endorsement

Description: The system shall allow users to endorse or remove endorsements on skills listed on a friend's profile. Self-endorsement is prohibited. Each endorser-skill-user combination is unique in skill_endorsements.

Rationale: Peer validation adds social proof to skill claims, improving search result quality.

Fit Criterion: Tapping the endorse button on a friend's skill inserts or deletes the matching row in skill_endorsements. The endorsement count updates immediately. Endorsing a non-friend is blocked at the application layer.

FR-09 — Notification System

Description: The system shall generate in-app notifications for: friend request received, friend request accepted, team join request received (admin), team join request accepted (member), task assigned, and meeting scheduled.

Rationale: Keeps users informed of actions requiring their attention, driving re-engagement.

Fit Criterion: Each triggering action inserts a row in the notifications table for the target user. The notification center badge updates in real-time via Supabase Realtime subscription.

FR-10 — Internationalization

Description: The system shall support Turkish and English languages, selectable from Settings. The selected locale shall persist across app restarts via SharedPreferences.

Rationale: The user base spans Turkish and English speakers; a bilingual app broadens accessibility.

Fit Criterion: After selecting English in Settings and restarting the app, all UI strings are displayed in English. No regression to Turkish occurs without a user action.

11. Data Requirements

The following tables constitute the primary data model in Supabase PostgreSQL. Row-Level Security is enabled on every table; users can only read and write rows they are authorized for.

Table	Key Columns	Purpose
profiles	id (FK → auth.users), username, full_name, avatar_url, bio, skills[], looking_for[], department, school, degree, education_year, open_to_work, is_profile_public, show_email, notif_*	Stores all user profile and preference data
friend_requests	id, requester_id (FK → profiles), addressee_id (FK → profiles), status (pending/accepted/rejected), request_type	Manages social connections and team invitations
teams	id, admin_id (FK → profiles), name, description, required_roles[], required_skills[], max_members, created_at	Represents project teams
team_members	id, team_id (FK → teams), user_id (FK → profiles), role	Junction table for team membership
team_requests	id, team_id, user_id, status (pending/approved/rejected)	Tracks join applications
team_tasks	id, team_id, title, description, status, due_date, subtasks (JSONB), created_by, created_at	Task records for the Kanban board
team_task_assignees	task_id, user_id	Junction table linking tasks to assigned users
team_meetings	id, team_id, created_by, title, description, meeting_date, meeting_link, created_at	Scheduled team meetings
skill_endorsements	id, user_id (endorsee), endorser_id, skill_name, created_at	Skill endorsement records
notifications	id, user_id (recipient), type, title, body, is_read, created_at	In-app notification records
reviews	id, reviewer_id, reviewed_id, rating (1–5), comment, created_at	Peer reviews and ratings

Key Relationships:

- profiles is the central entity; almost all tables reference it via foreign key.
- teams → team_members (one-to-many); team_tasks → team_task_assignees (many-to-many via junction).
- All child records of a team are cascade-deleted when the team is removed.

12. Dependability Requirements

12a. Reliability

Description: The application shall handle network errors, Supabase PostgrestException errors, and authentication failures gracefully without crashing.

Rationale: Student users are on mobile networks; intermittent connectivity is common.

Fit Criterion: All await calls to Supabase are wrapped in try/catch blocks. On failure, the UI displays a localized error message and the app remains functional. No uncaught exceptions appear in production builds.

12b. Availability

Description: The system shall target 99.5% monthly uptime. Planned maintenance windows shall be scheduled during off-peak hours (02:00–04:00 local time on weekdays).

Fit Criterion: Supabase SLA covers infrastructure uptime. Application-level outage is monitored via Supabase dashboard alerts. Any unplanned downtime exceeding 4 hours per month is flagged for review.

12c. Robustness

Description: In the event of a failed database migration or schema change, the development team shall execute a rollback to the last stable schema snapshot within 2 hours.

Fit Criterion: The supabase/ directory in the repository contains migration SQL files. A rollback SQL file is prepared before any schema change. Migrations are validated in a staging project before production deployment.

12d. Safety-Critical Requirements

TalentMesh does not control physical hardware or life-critical systems. System failures result in inconvenience, not physical danger. Data loss is the highest-severity risk; Supabase automatic daily backups mitigate this. No safety-critical classification is required.

III. Non-Functional Requirements

13. Maintainability, Supportability, and Adaptability

Update Schedule: Application updates will be released on a bi-weekly sprint cycle during active development. Post-launch maintenance follows a monthly cadence.

Cross-Platform Support: The Flutter codebase targets Android (primary), iOS, and Web. Platform-specific code is isolated in conditional blocks or plugin configurations.

Scalability: The Supabase free tier supports up to 60 concurrent connections and 500 MB storage, adequate for a pilot of up to 1,000 users. Scaling to a paid plan unlocks connection pooling (PgBouncer) without requiring code changes.

Code Maintainability: Services (AuthService, ProfileService, TeamService, NotificationService) are decoupled from UI widgets. Each service is a single-responsibility Dart class. Future refactoring to Riverpod state management is planned for v2.

Localization Maintainability: Adding a new language requires only a new JSON file under assets/translations/ and a new Locale entry in main.dart. No widget code changes are needed.

14. Security Requirements

Access Control

- Unauthenticated requests to any Supabase table are blocked by Row-Level Security (RLS) policies.
- The AuthGate widget checks `supabase.auth.currentUser` on startup and on every `AuthChangeEvent`; unauthenticated users cannot navigate past the login screen.
- Team-level actions (approve/reject requests, create tasks, delete team) are enforced both at the application layer (UI shows controls only to admin) and at the database layer (RLS policies check `admin_id = auth.uid()`).

Data Integrity

- PostgreSQL enforces foreign key constraints across all tables. Cascade deletes are configured for child records.
- The `skill_endorsements` table has a unique constraint on `(user_id, endorser_id, skill_name)` to prevent duplicate endorsements.

Privacy

- Passwords are never stored in the application layer; Supabase Auth handles bcrypt hashing internally.
- The `profiles` table has an `is_profile_public` flag; private profiles are hidden from search results via RLS policy.
- The `show_email` flag controls whether a user's email is exposed on their profile card.
- Avatar images are stored under UUID-based filenames to avoid exposing personally identifying information.

Compliance

KVKK (Turkish Personal Data Protection Law) and GDPR compliance requires: a privacy policy consent screen at registration (planned for v1.1), data export capability (planned), and account deletion with cascade data removal (partially implemented via team cascade deletes).

Injection and Virus Protection

All database interactions use the Supabase SDK's parameterized query builder, eliminating SQL injection risk. Uploaded avatar images are validated by file extension in the Flutter client before upload.

15. Usability and Humanity Requirements

Ease of Use: The application follows Material Design 3 conventions. Primary navigation is via a bottom navigation bar with clearly labeled icons.

Learning Curve: A new user should be able to complete registration, fill out a basic profile, and send their first friend request within 15 minutes of first launch, without consulting documentation.

Accessibility: The app uses minimum 48×48 dp tap targets, readable font sizes (minimum 13 sp via Google Fonts Inter), and high-contrast color tokens. Full screen reader (TalkBack/VoiceOver) support is deferred to v2.

Documentation: An in-app Help Center screen provides answers to common questions. A developer-facing README is maintained in the repository root.

Error Messages: All error states display user-friendly localized messages rather than raw exception text.

16. Look and Feel Requirements

Visual Style: The UI follows a clean, LinkedIn-inspired professional aesthetic adapted for a younger audience. Dual light/dark themes are implemented (AppTheme.lightTheme / AppTheme.darkTheme), toggleable by the user.

Typography: Google Fonts Inter is used throughout for its modern, highly legible sans-serif appearance at all sizes.

Color Palette: The primary accent color is teal-blue (#4A7C82), conveying trust and professionalism. Secondary accent colors for team cards cycle through a curated 8-color palette for visual variety.

Feel: Smooth navigation transitions, responsive loading skeleton states (shimmer-style placeholders), and tactile feedback on interactive elements give the app a polished feel.

Theme Mode: The ThemeProvider reacts to user changes instantly without restarting the app.

17. Operational and Environmental Requirements

External Integrations

- **Supabase:** Primary backend for auth, database, storage, and real-time (mandatory).
- **Google Fonts CDN:** Font assets fetched at runtime via google_fonts package; cached after first load.
- **URL Launcher:** Meeting links and external URLs opened in system browser via url_launcher ^6.3.2.

- **Image Picker:** Device camera and gallery access for avatar uploads via image_picker ^1.2.1.
- **QR Flutter:** QR code generation for sharing profile links via qr_flutter ^4.1.0.

Release Plan

Phase	Description	Target
v1.0 Alpha	Core auth, profile, friends, feed, teams, tasks, meetings	June 2026
v1.1 Beta	Privacy policy screen, KVKK consent, push notifications (FCM)	August 2026
v2.0	Riverpod state management, full accessibility, account export/delete	December 2026

18. Cultural, Political, and Legal Requirements

Cultural Compatibility: The app is designed primarily for Turkish university culture but supports English to accommodate international students. Date and time formats adapt to locale settings.

KVKK Compliance: Users in Turkey are subject to KVKK (Law No. 6698). A consent screen explaining data collection purposes and data subject rights (access, correction, deletion) must be in place before any public release, planned for v1.1.

GDPR: European users have equivalent rights under GDPR. The Supabase EU region option will be evaluated if the user base expands to Europe.

App Store Policies: Avatar images submitted by users are subject to Apple App Store and Google Play content policies. Moderation tooling is deferred to a future release.

Industry Standards: Passwords are managed by Supabase Auth (bcrypt hashing, JWT tokens). API communication is TLS-encrypted. No payment processing is involved in v1.

IV. Design & Test Plans

19. System Design Goals

Code Readability: Dart files are organized by layer (screens/, core/services/, models/, providers/). Each service class has a single, well-defined responsibility.

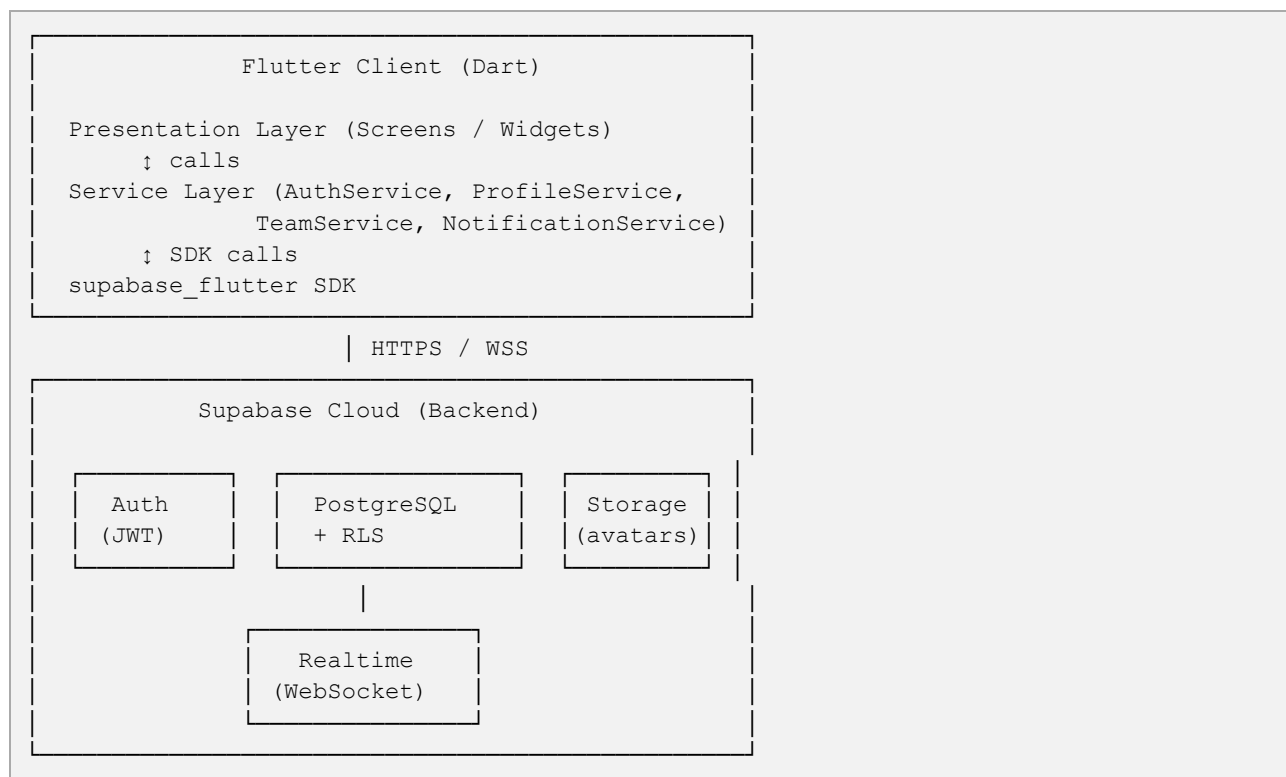
Speed: Supabase queries target sub-3-second response times on a 4G connection. Expensive queries batch sub-queries to minimize round trips.

Practicality: Features are prioritized by user impact. Complex features (real-time chat, full search indexing) are implemented pragmatically using available Supabase primitives rather than custom infrastructure.

Testability: Business logic is extracted into service classes, making it possible to mock Supabase responses in unit tests.

20. Software Architecture

TalentMesh follows a Client-Server architecture with a BaaS (Backend-as-a-Service) backend. The architecture is layered as follows:



Key Architectural Decisions:

- No custom server: Supabase eliminates the need for a Node.js/Python backend, reducing infrastructure cost to zero for the pilot phase.

- RLS as security layer: Row-Level Security policies serve as the authorization layer, complementing application-layer access controls.
- StatefulWidget + Service Pattern: Screens are StatefulWidget classes that instantiate service objects in initState and call async methods.
- Real-time via Streams: TeamService.getUserTeamsStream() uses Supabase's .stream() API to push updates to the UI without polling.

21. Test Plans

Test Philosophy

Testing follows a Pass/Fail gate model. A feature is considered complete when all its defined test cases pass. A module with failing tests is placed in a suspended state and reverted to the last passing commit before any release.

Test Levels

Level	Tool	Scope
Unit Tests	flutter_test	Service method logic, model fromJson/toJson
Widget Tests	flutter_test	Individual screen rendering, form validation
Integration Tests	Manual / flutter_driver	End-to-end flows on a real Android emulator

Pass/Fail Criteria

Criterion	Pass	Fail
Authentication flow	User logs in, reaches feed, JWT is valid	Login fails, app crashes, or JWT is null
Profile update	Field change persists to Supabase within 5 seconds	Data not persisted, or wrong user's row updated
Team creation	Team appears in My Teams immediately	Team not visible, or admin_id incorrect
Smart match	Returns ≥ 1 result for user with matching skills	Returns empty for a user with matching skills
Task Kanban	Status change persists and UI updates without full refresh	Status reverts, or wrong task updated
Notification delivery	Notification row inserted within 3 seconds of triggering action	No row inserted, or wrong recipient
Language switch	All strings change to selected language instantly	Any hardcoded string remains in previous language

Error Recovery Procedure

1. If a test suite fails during a release candidate build, the release is blocked.
2. The failing module is identified and its last passing commit is checked out.
3. The fix is implemented on a feature branch, tests are re-run, and a pull request is opened.
4. Only after all tests pass on the feature branch is it merged to main and the release re-attempted.

V. Project Issues

22. Open Issues and Risks

#	Issue / Risk	Impact	Status	Proposed Resolution
1	Push Notifications (FCM) not implemented	Users miss time-sensitive alerts when app is closed	Deferred to v1.1	Integrate firebase_messaging; configure FCM via Supabase Edge Functions
2	Offline Mode not supported	App is unusable without internet; poor UX on weak connections	Deferred to v2.0	Evaluate drift (SQLite) local cache with sync-on-reconnect logic
3	Supabase Free Tier Limits (60 connections, 500 MB)	Exceeded limits cause HTTP 503 errors for all users	Monitor via Supabase dashboard	Upgrade to Supabase Pro (\$25/month) when active users exceed 200
4	No Content Moderation	Users may post inappropriate profile content or team descriptions	Not addressed in v1	Add report/flag functionality and admin review queue in v1.1
5	Calendar Integration missing	Meeting dates not synced to device calendar	Deferred	Integrate add_2_calendar package in v1.1
6	KVKK / GDPR Consent Screen absent	Legal compliance gap before public release	In progress (v1.1)	Add consent dialog at first launch; implement data deletion endpoint
7	Full-text Search Performance	Client-side filtering may slow down at scale	Acceptable at pilot scale	Migrate to Supabase pg_trgm full-text search index for v2.0
8	State Management Scalability	Local StatefulWidget state causes redundant Supabase calls	Acceptable at current scale	Migrate to Riverpod in v2.0 for global state caching
9	Time Constraint	Academic deadline limits feature scope; several features postponed	Active	Prioritized backlog maintained; deferred features documented in this report
10	Server Cost Risk	Production Supabase costs unpredictable with user growth	Low for pilot	Set Supabase billing alerts; evaluate self-hosted Supabase for scale-out

VI. Glossary

Authentication: The process of verifying the identity of a user, typically via email and password, before granting access to protected resources.

BaaS (Backend-as-a-Service): A cloud service model providing ready-made backend infrastructure (database, auth, storage, real-time) without requiring a custom server.

Dart: The programming language used to build Flutter applications.

Flutter: An open-source UI toolkit from Google for building natively compiled, cross-platform applications from a single codebase.

JWT (JSON Web Token): A compact, URL-safe token used to securely transmit authentication information between the client and Supabase.

Kanban Board: A visual workflow management tool organizing tasks into columns representing stages of completion (To-Do, In Progress, Done).

KVKK: Kişisel Verilerin Korunması Kanunu — the Turkish Personal Data Protection Law (Law No. 6698), governing the collection and processing of personal data in Turkey.

Open to Work: A profile flag indicating that a user is actively looking for new project collaborators or employment opportunities.

PostgreSQL: An open-source relational database management system used as the primary data store in Supabase.

Profile: A user's public-facing data record stored in the profiles table, encompassing personal details, skills, and preferences.

RLS (Row-Level Security): A PostgreSQL feature that restricts which rows a user can view or modify, enforced at the database level.

Skill Endorsement: A peer affirmation from one user to another confirming that the endorsee possesses a particular skill.

Smart Match: An algorithmic recommendation that surfaces teams whose required skills overlap with the current user's listed skills.

Supabase: An open-source Firebase alternative providing PostgreSQL, authentication, real-time subscriptions, and file storage as managed cloud services.

Team Admin: The user who created a team and holds elevated permissions within that team's context.

WebSocket: A communication protocol providing full-duplex channels over a single TCP connection, used by Supabase Realtime for push updates.

VII. References / Bibliography

- [1] Flutter Documentation. (2024). Flutter – Build apps for any screen. <https://docs.flutter.dev>
- [2] Supabase Documentation. (2024). Supabase Docs. <https://supabase.com/docs>
- [3] Dart Documentation. (2024). Dart programming language. <https://dart.dev>
- [4] PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/>
- [5] easy_localization package. (2024). Flutter Internationalization. https://pub.dev/packages/easy_localization
- [6] Material Design 3. (2024). Material Design Guidelines. <https://m3.material.io>
- [7] Turkish Personal Data Protection Authority. (2016). Law No. 6698 on Protection of Personal Data (KVKK). <https://www.kvkk.gov.tr>
- [8] European Parliament. (2016). Regulation (EU) 2016/679 (GDPR). <https://gdpr-info.eu>